

Online Collaborative VLM Inference over Solar-Powered LEO Satellite Networks via Robust Lyapunov Optimization

Junfei Zhan, Tengjiao He

Abstract—This paper considers online collaborative Vision-Language Model (VLM) inference over solar-powered Low Earth Orbit (LEO) satellite networks.

Index Terms—LEO satellites, VLM inference, energy harvesting, model splitting, Lyapunov optimization, online scheduling.

I. INTRODUCTION

Low Earth Orbit (LEO) satellite constellations are evolving from communication relays into autonomous Earth-observation platforms [1]. Performing AI inference directly on board, rather than downlinking raw imagery, enables near real-time analysis of remote sensing data [2], [3]. Early on-board systems relied on convolutional neural networks (CNNs) trained on fixed datasets [4], yet satellite imagery is subject to significant *data drift*, including sensor degradation, seasonal illumination changes, and novel land-cover patterns, which degrades CNN accuracy after deployment. Vision-Language Models (VLMs), particularly the Vision Transformer (ViT) family [5], [6], offer a promising alternative: pre-trained ViT-based VLMs exhibit strong zero-shot generalization [7], making them well-suited for long-duration satellite missions where retraining is impractical.

However, a single satellite rarely has enough memory to host a large VLM [8]. A practical solution is *collaborative inference*: the VLM is split across multiple satellites, each executing a subset of layers and transmitting intermediate activations via inter-satellite links (ISLs). Several recent works have demonstrated this idea on terrestrial edge devices [9], [10] and, more recently, LEO constellations [11]. This introduces a joint optimization problem, namely which VLM configuration to deploy on each satellite, how to offload sensing tasks through the ISL network, and when to schedule inference, placing LEO satellites in the broader paradigm of orbital edge computing but under structural constraints that are far more severe than terrestrial edge settings.

These constraints are qualitatively different from those in terrestrial edge computing. As illustrated in Fig. 1, the LEO setting imposes four structural constraints that render existing collaborative inference and energy-harvesting schedulers inapplicable. The two most critical are energy intermittency and peak power safety. (i) *Periodic eclipse*. Each ~ 95 -minute orbit alternates between sunlit ($\sim 60\%$) and eclipse ($\sim 40\%$) phases [12]; during eclipse, solar energy drops to zero and the battery must sustain all computation. Standard Lyapunov schedulers require a non-positive drift at every slot, a condition

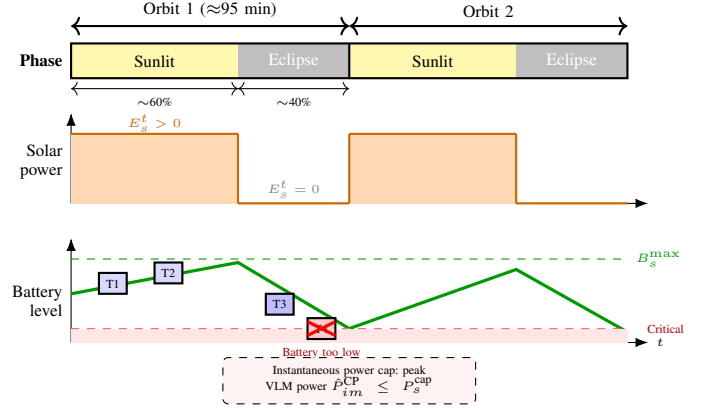


Fig. 1. Energy challenges in solar-powered LEO satellite inference. A satellite at 550 km altitude experiences $\sim 60\%$ sunlit and $\sim 40\%$ eclipse per 95-minute orbit. Solar power drops to zero during eclipse, forcing the battery to sustain all computation. Tasks T1–T3 execute normally, but T4 is infeasible due to depleted battery. Each slot must respect an instantaneous power cap P_s^{cap} .

that is structurally violated during zero-harvest intervals; existing LEO collaborative inference work [11] sidesteps the issue entirely by assuming unlimited grid-like energy. (ii) *Instantaneous power cap*. Satellite hardware enforces a strict per-slot power ceiling; exceeding it triggers immediate shutdown. VLM inference exhibits sharp power transients during model loading, with instantaneous GPU power reaching $2\text{--}3\times$ the steady-state level (see Section III-D for measurements). All existing EH scheduling methods [13]–[15] model only average energy consumption, so their schedules can be energy-feasible yet violate peak power limits on real hardware. Two additional constraints further distinguish the LEO setting. (iii) *Dynamic ISL topology*: inter-plane links appear and disappear as orbital planes shift [16], so static offloading policies become infeasible when the target satellite is unreachable. (iv) *Quality-energy coupling*: larger VLMs yield higher accuracy but draw more power, requiring joint model selection and energy co-optimization that is absent from existing collaborative inference frameworks for LEO.

While [11] addresses collaborative VLM inference over LEO and [17] addresses solar-powered edge scheduling, neither handles the joint coupling of eclipse-induced energy intermittency and hardware peak-power caps that is central to on-orbit VLM deployment. Terrestrial collaborative inference methods [18], [19] and on-device LLM optimizations [20], [21] assume unlimited energy and static networks, making

them inapplicable to the LEO setting.

To the best of our knowledge, this is the first work to jointly address these coupled constraints. We propose a two-timescale online framework: an offline stage determines VLM deployment across the constellation, and an online Lyapunov-based scheduler makes per-slot offloading and model-selection decisions using only current state, maximizing the minimum long-term inference quality across sensing sources (max-min fairness). Two technical components enable this framework: a robust Lyapunov analysis based on input-to-state stability (ISS) [22] that preserves queue stability through zero-harvest eclipse intervals, and calibrated peak-power bounds obtained from offline hardware profiling that render per-slot power feasibility checkable in closed form.

The main contributions are:

- C1.** *First formulation* of energy-aware collaborative VLM inference over solar-powered LEO satellite networks, jointly capturing two-stage pipeline splitting across satellites, instantaneous peak-power caps, orbital eclipse dynamics, dynamic ISL topology, and max-min quality fairness.
- C2.** *A two-timescale robust Lyapunov framework* comprising an offline ILP for VLM deployment (including pipeline stage pairing) and a lightweight per-slot online scheduler, with calibrated power bounds that make peak-power feasibility checkable per slot.
- C3.** *Rigorous theoretical guarantees:* an $O(1/V)$ optimality gap, mean-rate constraint satisfaction under a Slater condition, and robust stability during eclipse via an ISS-based analysis that extends single-queue routing results to coupled energy–power queues.
- C4.** *Empirical validation* on EuroSAT [23] and RESISC45 [24] with ViT-B/16 through ViT-G/14 at multiple quantization levels, showing that the proposed framework achieves zero peak-power violations while maintaining min-quality within a small gap of the offline MILP benchmark, compared to baselines that ignore one or more of the four constraints.

The remainder of this paper is organized as follows. Section II reviews related works. Section III presents the system model and problem formulation. Section IV describes the robust Lyapunov online optimization framework. Section V provides theoretical analysis. Section VI reports evaluation results, and Section VII concludes the paper.

II. RELATED WORKS

Our research overlaps with works that aim to (i) schedule AI inference tasks on energy-harvesting edge devices, (ii) split and distribute large models across multiple nodes for collaborative inference, and (iii) model the energy and power dynamics of neural network inference.

A. EH-Aware Edge AI Scheduling

Mobile edge computing with energy harvesting (EH) has been studied extensively [13]. Mei et al. [14] and Luo et al. [17] optimize task offloading and workload scheduling for solar-powered edge servers, while Khoshsir et al. [15] extend

EH-aware scheduling to decentralized LLM inference. For terrestrial EH networks, the solar energy arrival is commonly modeled by HMMs [34]; in LEO, however, solar energy is governed by deterministic orbital mechanics rather than atmospheric conditions, requiring a different modeling approach.

In the LEO satellite domain, a range of works address computation offloading and resource allocation: mobility-aware offloading [29], game-theoretic UAV-LEO offloading [31], energy-constrained satellite-terrestrial scheduling [25], inter-satellite cooperative caching [32], LLM-assisted resource allocation [27], satellite-ground cooperation platforms [26], two-timescale service deployment [28], and semantic communication for offloading [30]. However, none of these works jointly address collaborative VLM inference with model splitting, energy harvesting, or instantaneous power cap constraints.

B. Collaborative Inference and Model Splitting

When a model exceeds the memory of a single device, a natural solution is to split it across multiple nodes. The Vision Transformer (ViT) [5] is particularly amenable to layer-wise partitioning due to its uniform block structure. On terrestrial edge devices, DeViT [9] decomposes large ViTs for collaborative inference, EdgeShard [18] partitions LLMs across heterogeneous devices, Dai et al. [10] jointly optimize device placement and model partitioning with an online algorithm, Tpi-LLM [19] serves 70B-scale LLMs via tensor parallelism, and Lyu et al. [8] analyze the model-size vs. communication overhead trade-off. For on-device inference, EdgeLLM [20] employs speculative decoding and He et al. [21] study LLM offloading in cloud-edge computing.

In the satellite domain, Zhang et al. [11] split a ViT across multiple LEO satellites with pipeline parallelism and adaptive activation compression, validating on EuroSAT [23] and RESISC45 [24]. However, their formulation only minimizes inference delay and has no energy harvesting model, battery dynamics, or instantaneous power cap constraint. Complementary to model splitting, SLICER [35] proposes training-free activation compression that reduces transmitted volume by up to $10\times$.

C. Energy and Power Modeling for AI Inference

Accurate energy and power models are essential for energy-aware scheduling. Kasioulis et al. [36] benchmarked energy modeling of inference workloads with AI accelerators at the edge, revealing significant variance in power consumption across model architectures. Husom et al. [37] evaluated quantized LLMs for energy efficiency, and Xu et al. [38] proposed an energy-aware LLM inference framework for resource-constrained devices.

A key observation is that neural network inference exhibits *phase-wise* power dynamics: GPU power ramps up during model loading, reaches a plateau during steady-state computation, and decays upon completion. In this work, we profile each VLM configuration on the target hardware to extract calibrated instantaneous power bounds, which enter the optimization as offline-computed parameters.

TABLE I
A COMPARISON OF RELATED WORKS.

Ref.	Energy Harvesting	Model Splitting	Activation Compress.	Power Cap Constraint	Online Algorithm	Max-Min Quality	LEO Satellite	VLM
[13]								
[14], [15]	✓							
[17]	✓				✓			
[25]	✓				✓		✓	
[26], [27]							✓	✓
[22], [28], [29]					✓		✓	
[30]–[32]							✓	
[8], [9]		✓						✓
[10]		✓			✓			
[18]		✓						
[11]		✓	✓				✓	✓
[20]								✓
[21]					✓			✓
[33]					✓			
Our Work	✓	✓	✓	✓	✓	✓	✓	✓

D. Lyapunov Optimization for Online Resource Management

Lyapunov optimization [39] decomposes long-term constrained problems into per-slot subproblems via virtual queues and drift-plus-penalty minimization, and has been applied to wireless resource allocation [40], edge-cloud migration [41], and Lyapunov-guided deep RL for MEC [33]. In the LEO domain, Huang et al. [22] introduced a relaxed stability condition based on input-to-state stability (ISS), proving that the system remains stable as long as the overall Lyapunov drift trend is downward, even under temporary violations caused by bursty traffic or intermittent connectivity. This robust framework is relevant to our setting, where eclipse periods disrupt energy supply.

E. Discussion

Table I summarizes the comparison. No prior work jointly addresses collaborative VLM inference with model splitting, solar energy harvesting with eclipse dynamics, instantaneous peak-power caps, dynamic ISL topology, and max-min quality fairness within an online optimization framework. Our work fills this gap by unifying all eight dimensions in Table I.

III. SYSTEM MODEL AND PROBLEM FORMULATION

We consider a LEO satellite constellation performing on-orbit Earth observation tasks using VLMs. Fig. 2 illustrates the two collaborative inference topologies under sunlit and eclipse phases, and Table II summarizes the key notations used throughout this paper.

A. Network and VLM Configuration Model

The constellation comprises two types of satellites: a set $\mathcal{S}$ of computing satellites and a set $\mathcal{I}$ of sensing satellites. This partition reflects the heterogeneous nature of LEO constellations, where earlier-generation satellites with limited onboard compute serve as sensing nodes, while newer satellites with dedicated accelerators host VLM inference workloads. Each computing satellite $s \in \mathcal{S}$ is equipped with a solar panel, a rechargeable battery with capacity $B_s^{\max}$, and an onboard computing platform with RAM capacity $R_s^{\max}$. Each sensing

satellite $i \in \mathcal{I}$ generates observation tasks at rate λ_i ; let $a_i^t \in \{0, 1\}$ indicate whether source i has a pending task at slot t , with data size D_i . We consider a planning horizon of $|\mathcal{T}|$ discrete time slots, each of duration τ . Each satellite hosts at most one VLM configuration $y_{sm} \in \{0, 1\}$ for the entire horizon:

$$\sum_{m \in \mathcal{M}} y_{sm} \leq 1, \quad \forall s \in \mathcal{S}, \quad (1)$$

$$\sum_{m \in \mathcal{M}} y_{sm} \cdot R_m \leq R_s^{\max}, \quad \forall s \in \mathcal{S}. \quad (2)$$

The VLM configuration set $\mathcal{M} = \mathcal{M}_S \cup \mathcal{M}_F \cup \mathcal{M}_R$ contains standalone configurations ($\mathcal{M}_S$, full model on one satellite), front-stage configurations ($\mathcal{M}_F$, first L_m transformer blocks), and rear-stage configurations ($\mathcal{M}_R$, remaining blocks). A bijection $\pi : \mathcal{M}_F \rightarrow \mathcal{M}_R$ pairs each front stage with its rear counterpart. Each configuration m specifies the VLM architecture, quantization level, layer range, and activation compression bit-width for ISL transmission. Because ViT blocks are identical, the intermediate activation size is constant regardless of the split point [11], and activation transfer at laser-ISL rates is negligible compared with inference latency. A task executes on satellite s only if $y_{sm} = 1$, and for a split pipeline both stages must be deployed:

$$x_{ism}^t \leq y_{sm}, \quad \forall i, s, m, t, \quad (3)$$

$$y_{sm} \leq \sum_{s' \in \mathcal{S}} y_{s', \pi(m)}, \quad \forall s \in \mathcal{S}, \forall m \in \mathcal{M}_F. \quad (4)$$

For each pair (i, m) , the expected energy $\mathbb{E}[E_{im}]$, execution time $\mathbb{E}[T_{im}]$, calibrated power bound $\hat{P}_{im}^{\text{CP}}$, and required slots $N_{im} = \lceil \mathbb{E}[T_{im}] / \tau \rceil$ are computed offline. The inference quality Q_{im} denotes the task-specific accuracy of configuration m on tasks from source i (e.g., determined by land-cover category); it equals the full-pipeline accuracy for $m \in \mathcal{M}_S \cup \mathcal{M}_R$ and is zero for $m \in \mathcal{M}_F$, since quality is attributed entirely to the rear stage $\pi(m)$.

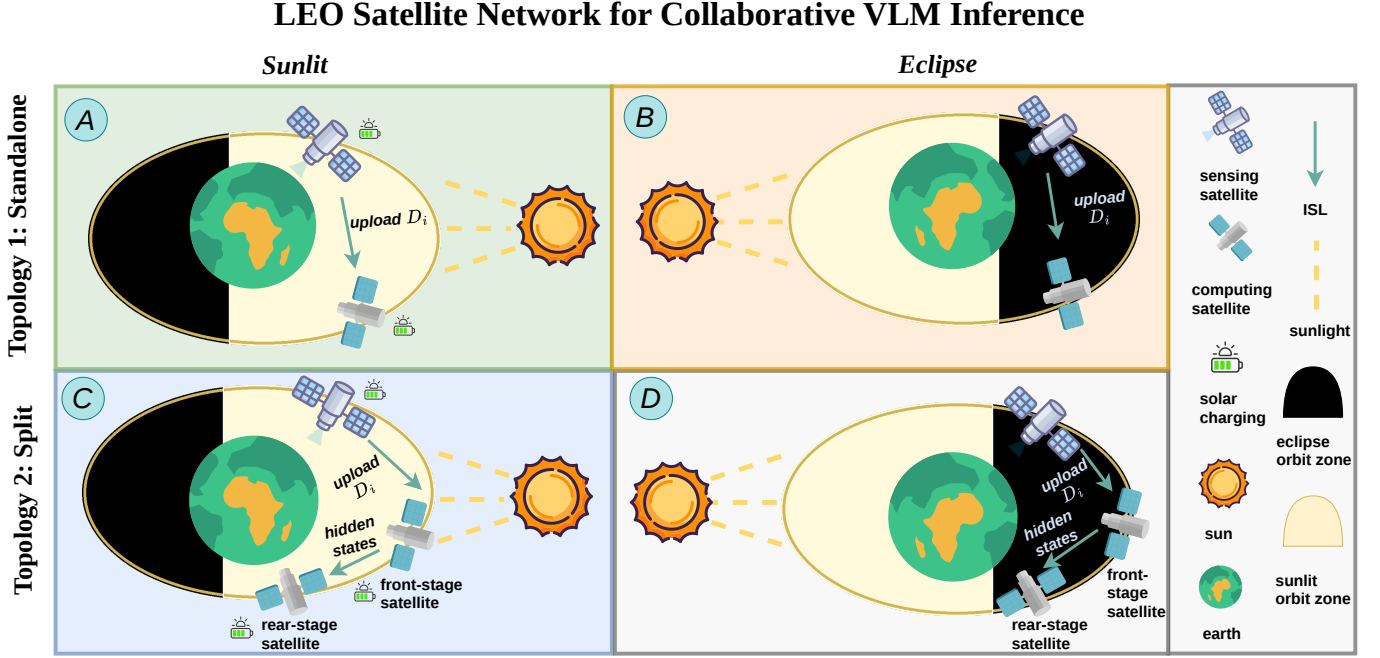


Fig. 2. System overview of the LEO satellite network for collaborative VLM inference under sunlit and eclipse phases. (A) Standalone topology, sunlit: sensing satellite uploads D_i via ISL to a computing satellite running the full VLM. (B) Standalone, eclipse: battery sustains inference without solar input. (C) Split pipeline, sunlit: D_i is forwarded to a front-stage satellite, which passes intermediate hidden states via ISL to a rear-stage satellite. (D) Split pipeline, eclipse: both computing satellites rely on battery power; ISL connectivity must be maintained at the activation handoff slot $t + N_{im}$.

B. Communication and Task Offloading

Following the standard +Grid ISL topology adopted in LEO constellations [16], [22], each satellite maintains four inter-satellite links: two *intra-plane* links connecting the adjacent satellites in the same orbit, and two *inter-plane* links connecting satellites in neighboring orbital planes. Intra-plane neighbors are fixed because satellites in the same orbit share identical ground tracks. Inter-plane neighbors, however, change over time as satellites in different orbital planes move at different angular velocities; within each slot the topology is treated as static, but the inter-plane connectivity may differ across slots. When no ISL path exists between source i and satellite s at slot t , we set $C_{is}^t = 0$.

Modern LEO constellations increasingly employ free-space optical (laser) ISLs [16], which operate with a clear line-of-sight path and no multipath scattering. We therefore adopt a free-space path loss (FSPL) model. The received signal-to-noise ratio between the sensing satellite of source i and computing satellite s in slot t is

$$\text{SNR}_{is}^t = \frac{P_{\text{tx}} G_{\text{tx}} G_{\text{rx}}}{k_B T_{\text{sys}} W} \cdot \left(\frac{c}{4\pi f d_{is}^t} \right)^2, \quad (5)$$

where P_{tx} is the transmit power, G_{tx} and G_{rx} are the transmit and receive antenna gains, k_B is the Boltzmann constant, T_{sys} is the system noise temperature, W is the bandwidth, c is the speed of light, f is the carrier frequency, and d_{is}^t is the distance between the two satellites in slot t . The inter-satellite

distance d_{is}^t varies over time due to orbital dynamics. The link capacity is

$$C_{is}^t = W \log_2(1 + \text{SNR}_{is}^t). \quad (6)$$

Let $\ell_{ss'}^t \in \{0, 1\}$ denote whether computing satellites s and s' are ISL-connected at slot t . This indicator is computable offline from orbital ephemeris data and is needed for activation handoff in pipeline execution (Section III-C).

Let $\alpha_{is}^t \in \{0, 1\}$ denote whether the task from source i is uploaded to satellite s in slot t . Each task is uploaded at most once, and the upload must fit within a single slot:

$$\sum_{s \in \mathcal{S}} \sum_{t \in \mathcal{T}} \alpha_{is}^t \leq 1, \quad \forall i \in \mathcal{I}, \quad (7)$$

$$\alpha_{is}^t \cdot D_i \leq C_{is}^t \cdot \tau, \quad \forall i \in \mathcal{I}, \forall s \in \mathcal{S}, \forall t \in \mathcal{T}. \quad (8)$$

C. Task Scheduling

Let $x_{ism}^t \in \{0, 1\}$ denote whether the task from source i starts execution on satellite s with configuration m at slot t . Each task is processed at most once; to avoid double-counting, we restrict the at-most-once constraint to standalone or front-stage starts, since the rear-stage execution is jointly determined by the pipeline coupling and precedence constraints (14)–(15). Specifically, x_{ism}^t remains a decision variable because multiple satellites may host the rear-stage configuration, and

the scheduler must select which one to use subject to ISL connectivity (16):

$$\sum_{s \in \mathcal{S}} \sum_{m \in \mathcal{M}_S \cup \mathcal{M}_F} \sum_{t \in \mathcal{T}} x_{ism}^t \leq 1, \quad \forall i \in \mathcal{I}. \quad (9)$$

Execution of a standalone or front-stage configuration can start only after the task data has been uploaded to the same satellite s , ensuring that the upload destination and the execution host coincide:

$$x_{ism}^t \leq \sum_{k=1}^t \alpha_{is}^k, \quad \forall i, s, m \in \mathcal{M}_S \cup \mathcal{M}_F, t. \quad (10)$$

A rear-stage configuration receives intermediate activations via ISL rather than raw data, so its prerequisite is the precedence constraint (15) instead of (10).

Once started, a task runs non-preemptively for N_{im} consecutive slots. The execution indicator $z_{ism}^t \in \{0, 1\}$ is fully determined by the start variables:

$$z_{ism}^t = \sum_{t'=\max(1, t-N_{im}+1)}^t x_{ism}^{t'}, \quad \forall i, s, m, t. \quad (11)$$

At most one task per (satellite, configuration) at any time:

$$\sum_{i \in \mathcal{I}} z_{ism}^t \leq 1, \quad \forall s \in \mathcal{S}, \forall m \in \mathcal{M}, \forall t \in \mathcal{T}. \quad (12)$$

All tasks must complete within the horizon:

$$x_{ism}^t \cdot (t + N_{im} - 1) \leq |\mathcal{T}|, \quad \forall i, s, m, t. \quad (13)$$

For split configurations, starting a front stage commits the system to executing the corresponding rear stage. Specifically, if a front stage is scheduled, the corresponding rear stage must also be scheduled:

$$\sum_s \sum_t x_{is, \pi(m)}^t = \sum_s \sum_t x_{ism}^t, \quad \forall i \in \mathcal{I}, m \in \mathcal{M}_F. \quad (14)$$

The rear stage starts immediately after the front stage completes (activation transfer is negligible at laser-ISL rates):

$$x_{is', \pi(m)}^{t'} \leq \sum_{s \in \mathcal{S}} x_{ism}^{t'-N_{im}}, \quad \forall i, s', m \in \mathcal{M}_F, t'. \quad (15)$$

Since activations are transferred via ISL at handoff, the front- and rear-stage satellites must be ISL-connected at that moment:

$$x_{ism}^t + x_{is', \pi(m)}^{t+N_{im}} \leq 1 + \ell_{ss'}^{t+N_{im}}, \quad \forall i, s, s', m \in \mathcal{M}_F, t. \quad (16)$$

D. Energy and Resource Constraints

Each satellite s harvests solar energy. Unlike terrestrial settings where weather introduces multi-state stochasticity [34], LEO solar energy is governed by orbital mechanics: a satellite at altitude h follows a periodic orbit of period T_{orb} , spending a fraction f_{sun} in sunlight and $1 - f_{\text{sun}}$ in eclipse. Let $\delta_s^t \in \{0, 1\}$ denote the sunlit indicator of satellite s at slot t , where $\delta_s^t = 1$ if the satellite is in sunlight and $\delta_s^t = 0$ during eclipse; this

indicator is deterministic and computable offline from two-line element (TLE) data. We model the solar energy arrival as

$$E_s^t = P_{\text{solar}} \cdot (\eta_s + \epsilon_s^t) \cdot \tau \cdot \delta_s^t, \quad (17)$$

where P_{solar} is the solar irradiance at the panel, η_s is the panel efficiency, and $\mathbb{1}[\cdot]$ is the sunlit indicator determined by the orbital ephemeris. The sunlit/eclipse transition is periodic and predictable from two-line element (TLE) data; we add a small random perturbation $\epsilon_s^t \sim \mathcal{N}(0, \sigma_\eta^2)$ to η_s to account for attitude variations and panel degradation.

$$0 \leq \omega_s^t \leq E_s^t, \quad \forall s \in \mathcal{S}, \forall t \in \mathcal{T}, \quad (18)$$

$$b_s^t + \omega_s^t \leq B_s^{\max}, \quad \forall s \in \mathcal{S}, \forall t \in \mathcal{T}. \quad (19)$$

The battery level evolves as

$$b_s^{t+1} = b_s^t + \omega_s^t - \sum_{i,m} z_{ism}^t \cdot \frac{\mathbb{E}[E_{im}]}{N_{im}}, \quad \forall s, t. \quad (20)$$

The battery level is non-negative, and energy consumption cannot exceed available energy in any slot:

$$\sum_{i,m} z_{ism}^t \cdot \frac{\mathbb{E}[E_{im}]}{N_{im}} \leq b_s^t + \omega_s^t, \quad \forall s, t, \quad (21)$$

$$b_s^t \geq 0, \quad \forall s \in \mathcal{S}, \forall t \in \mathcal{T}. \quad (22)$$

The instantaneous power cap requires that the total power draw never exceeds the hardware limit:

$$\sum_{i,m} z_{ism}^t \cdot \hat{P}_{im}^{\text{CP}} \leq P_s^{\text{cap}}, \quad \forall s, t. \quad (23)$$

The calibrated power bound $\hat{P}_{im}^{\text{CP}}$ is obtained by offline hardware profiling: for each configuration m , we record the instantaneous power trace and extract a high quantile of the peak power distribution, since VLM inference exhibits phase-wise power dynamics that cause instantaneous power to significantly exceed the average.

E. Problem Formulation

Our objective is to maximize the minimum inference quality across all sensing sources. We jointly optimize (i) task offloading decisions, (ii) VLM configuration deployment, (iii) task scheduling, (iv) energy management, and (v) quality assurance.

Let $q_i^t = \sum_{s,m} x_{ism}^t \cdot Q_{im}$ denote the inference quality delivered to source i in slot t . Since $Q_{im} = 0$ for all $m \in \mathcal{M}_F$, quality is credited only when a standalone configuration completes or a rear stage starts, which is exactly when the full VLM inference result becomes available. Define the long-term average quality per source as $\bar{q}_i = \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \mathbb{E}[q_i^t]$. The stochastic optimization problem is:

$$\max_{\pi} \min_{i \in \mathcal{I}} \bar{q}_i \quad (\text{P})$$

s.t. (1)-(16),

$$\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \mathbb{E} \left[\sum_{i,m} z_{ism}^t \cdot \frac{\mathbb{E}[E_{im}]}{N_{im}} - \omega_s^t \right] \leq 0, \quad \forall s, \quad (24)$$

$$\sum_{i,m} z_{ism}^t \cdot \hat{P}_{im}^{\text{CP}} \leq P_s^{\text{cap}}, \quad \forall s, t, \quad (25)$$

where $\pi = [\alpha_{is}^t, y_{sm}, x_{ism}^t]$ denotes the collection of all decision variables (z_{ism}^t is determined by x via (11)). The objective maximizes the minimum long-term average quality across all sensing sources. Constraint (24) enforces a long-term average energy budget at each satellite, and (25) enforces the per-slot instantaneous power cap using calibrated power bounds. The energy and power sums in (24)–(25) naturally account for both standalone and pipeline executions: a satellite hosting a front stage incurs the front-stage energy and power during its execution window, while the rear-stage satellite incurs the corresponding rear-stage costs.

Problem (P) is a long-term stochastic optimization that couples decisions across time through battery dynamics, non-preemptive scheduling, and pipeline precedence constraints. It involves both binary decision variables ($\alpha_{is}^t, y_{sm}, x_{ism}^t$) and continuous state variables (b_s^t); when all future information is known, it reduces to a finite-horizon mixed-integer linear program (MILP). Solving the stochastic version directly requires full distributional knowledge and suffers from the curse of dimensionality, as the total number of binary variables scales as $O(|\mathcal{I}| \cdot |\mathcal{S}|^2 \cdot |\mathcal{M}| \cdot |\mathcal{T}|)$ (the $|\mathcal{S}|^2$ factor arises from pipeline satellite pairs). In the next section, we develop an online algorithm that avoids these difficulties.

IV. ROBUST LYAPUNOV ONLINE OPTIMIZATION

In this section, we design an online algorithm based on Lyapunov drift-plus-penalty [39] that decomposes the long-term problem (P) into per-slot subproblems without requiring future information. We further introduce a robust extension inspired by [22] to handle the intermittent energy supply caused by eclipse periods.

As a performance reference, we define an offline MILP benchmark that assumes full future knowledge. By introducing an auxiliary variable U to linearize the max-min objective, the offline formulation is:

$$\begin{aligned} \max_{\pi, U} \quad & U \quad (\mathbf{P}_{\text{off}}) \\ \text{s.t.} \quad & (1)-(16), \\ & U \leq \sum_{s,m,t} x_{ism}^t \cdot Q_{im}, \quad \forall i \in \mathcal{I}. \end{aligned} \quad (26)$$

This is a pure MILP because all energy and power parameters are precomputed offline. It serves as an upper bound on the achievable performance of any online algorithm. We use it in Section V to characterize the optimality gap.

A. Two-Timescale Decomposition

We observe that problem (P) has a natural two-timescale structure. The model deployment decision y_{sm} determines which VLM configuration is loaded on each satellite. Loading a model into GPU memory is costly, so y_{sm} remains fixed over the planning horizon. In contrast, the task offloading α_{is}^t and scheduling x_{ism}^t decisions must be made in each time slot under stochastic arrivals.

We exploit this structure by decomposing the problem into two levels.

1) *Upper Level — Offline Model Deployment*: Given the statistical characteristics of task arrivals and solar energy, we solve a small-scale integer program to determine the model deployment:

$$\begin{aligned} \max_{\{y_{sm}\}} \quad & \sum_{s \in \mathcal{S}} \sum_{m \in \mathcal{M}} y_{sm} \cdot \bar{Q}_m \quad (27) \\ \text{s.t.} \quad & \sum_{m \in \mathcal{M}} y_{sm} \leq 1, \quad \forall s \in \mathcal{S}, \\ & \sum_{m \in \mathcal{M}} y_{sm} \cdot R_m \leq R_s^{\max}, \quad \forall s \in \mathcal{S}, \\ & y_{sm} \leq \sum_{s'} y_{s', \pi(m)}, \quad \forall s \in \mathcal{S}, m \in \mathcal{M}_F, \end{aligned}$$

where $\bar{Q}_m = \frac{1}{|\mathcal{I}|} \sum_i Q_{im}$ is the average quality of configuration m across sources (note that $\bar{Q}_m = 0$ for $m \in \mathcal{M}_F$, so the objective naturally rewards deploying rear stages and standalone configurations, while the pipeline coupling constraint ensures that front stages are co-deployed). This problem has $|\mathcal{S}| \cdot |\mathcal{M}|$ binary variables and can be solved efficiently. Once y_{sm} is fixed, the set of feasible configurations at each satellite is determined for the entire horizon.

2) *Lower Level — Online Per-Slot Scheduling*: Given the model deployment $\{y_{sm}\}$, the remaining decisions α_{is}^t and x_{ism}^t are made online in each slot t using the Lyapunov drift-plus-penalty framework described below.

B. Virtual Queue Design

The key idea of Lyapunov optimization is to convert long-term constraints into virtual queues whose stability implies constraint satisfaction [39]. We introduce three types of virtual queues.

1) *Energy Virtual Queue*: For each satellite $s \in \mathcal{S}$, we define a virtual queue $Q_s^E(t)$ that tracks the cumulative energy deficit:

$$Q_s^E(t+1) = \max \left\{ Q_s^E(t) + \sum_{i,m} z_{ism}^t \frac{\mathbb{E}[E_{im}]}{N_{im}} - \omega_s^t, 0 \right\}, \quad (28)$$

where $Q_s^E(0) = 0$. When the energy consumed exceeds the energy harvested, the queue backlog grows; when solar energy is abundant, it decreases. Stabilizing $Q_s^E(t)$ in the mean-rate sense ensures the long-term energy budget constraint (24) is satisfied.

2) *Power Cap Virtual Queue*: For each satellite $s \in \mathcal{S}$, we define a virtual queue $Q_s^P(t)$ that tracks instantaneous power cap violations:

$$Q_s^P(t+1) = \max \left\{ Q_s^P(t) + \sum_{i,m} z_{ism}^t \hat{P}_{im}^{\text{CP}} - P_s^{\text{cap}}, 0 \right\}, \quad (29)$$

where $Q_s^P(0) = 0$. The calibrated power bound $\hat{P}_{im}^{\text{CP}}$ (obtained via offline profiling, see Section III-D) enters directly into this queue.

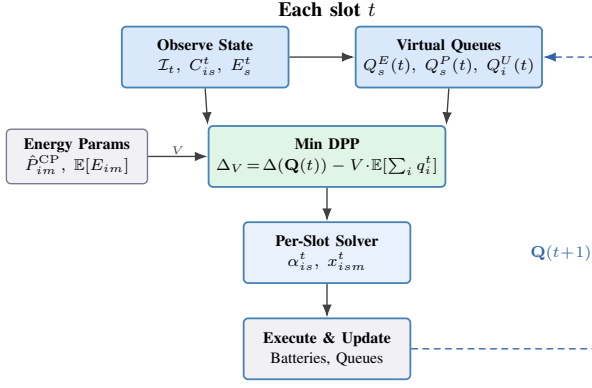


Fig. 3. Schematic of the online scheduling loop (lower level). In each slot t , the scheduler monitors the system state and virtual queue backlogs to formulate the drift-plus-penalty (DPP) objective. Using pre-computed energy parameters, the framework solves a lightweight per-slot optimization, executes the control decisions, and updates queue states for the subsequent slot.

3) *Quality Deficit Virtual Queue*: For each sensing source $i \in \mathcal{I}$, we define a virtual queue $Q_i^U(t)$ that tracks the gap between the target quality level U^{tgt} and the achieved quality:

$$Q_i^U(t+1) = \max\{Q_i^U(t) + a_i^t \cdot U^{\text{tgt}} - q_i^t, 0\}, \quad (30)$$

where $a_i^t \in \{0, 1\}$ is the task arrival indicator and U^{tgt} is an adjustable quality target. The arrival indicator ensures that the queue grows only when source i has a pending task; once the task is processed, $q_i^t = Q_{im}^t > 0$ provides a credit. Stabilizing this queue ensures that each source's average quality meets the target.

Fig. 3 illustrates the online scheduling loop.

C. Lyapunov Drift-Plus-Penalty

Define the aggregate virtual queue vector $\mathbf{Q}(t) = \{Q_s^E(t), Q_s^P(t), Q_i^U(t)\}_{s \in \mathcal{S}, i \in \mathcal{I}}$ and the quadratic Lyapunov function:

$$L(\mathbf{Q}(t)) = \frac{1}{2} \sum_{s \in \mathcal{S}} [(Q_s^E(t))^2 + (Q_s^P(t))^2] + \frac{1}{2} \sum_{i \in \mathcal{I}} (Q_i^U(t))^2. \quad (31)$$

The one-slot conditional Lyapunov drift is:

$$\Delta(\mathbf{Q}(t)) = \mathbb{E}[L(\mathbf{Q}(t+1)) - L(\mathbf{Q}(t)) \mid \mathbf{Q}(t)]. \quad (32)$$

By the standard Lyapunov technique, $\Delta(\mathbf{Q}(t))$ can be upper-bounded as follows. Using the fact that $(\max\{a+b, 0\})^2 \leq a^2 + b^2 + 2ab$ for all a, b , we obtain

$$\begin{aligned} \Delta(\mathbf{Q}(t)) &\leq B + \sum_s Q_s^E(t) \mathbb{E}[e_s^t - \omega_s^t \mid \mathbf{Q}(t)] \\ &\quad + \sum_s Q_s^P(t) \mathbb{E}[p_s^t - P_s^{\text{cap}} \mid \mathbf{Q}(t)] \\ &\quad + \sum_i Q_i^U(t) \mathbb{E}[a_i^t U^{\text{tgt}} - q_i^t \mid \mathbf{Q}(t)], \end{aligned} \quad (33)$$

where $e_s^t = \sum_{i,m} z_{ism}^t \mathbb{E}[E_{im}]/N_{im}$ is the energy consumed, $p_s^t = \sum_{i,m} z_{ism}^t \hat{P}_{im}^{\text{CP}}$ is the total power, $q_i^t = \sum_{s,m} x_{ism}^t Q_{im}^t$ is the quality delivered to source i , and B is a finite constant (see Section V).

The drift-plus-penalty function adds a penalty term weighted by the control parameter $V > 0$:

$$\Delta_V(\mathbf{Q}(t)) = \Delta(\mathbf{Q}(t)) - V \cdot \mathbb{E}\left[\sum_i q_i^t \mid \mathbf{Q}(t)\right]. \quad (34)$$

The penalty $\sum_i q_i^t$ is the total inference quality delivered in slot t . The max-min fairness is not enforced by the penalty but by the quality virtual queues: sources with large $Q_i^U(t)$ backlogs receive priority in the per-slot problem. The parameter V controls the trade-off between queue stability (constraint satisfaction) and quality throughput. A larger V pushes the scheduler to deliver more quality at the cost of larger queue backlogs.

D. Per-Slot Optimization

By the principle of opportunistic minimization [39], we minimize the right-hand side of the drift-plus-penalty bound in each slot t . For standalone and front-stage decisions, the subproblem depends only on the current virtual queue backlogs $\mathbf{Q}(t)$ and the current system state. For pipeline tasks, starting a front stage at slot t commits the rear stage to execute at a predictable future slot; because the ISL topology and satellite availability are computable from orbital ephemeris and the current z state, this limited lookahead is bounded by $\max_{i,m} N_{im}$ slots and does not require stochastic future information.

Given the model deployment $\{y_{sm}\}$ and the current virtual queue backlogs, the per-slot decision in slot t proceeds in two steps. First, all *mandatory* rear-stage tasks (whose front stages completed in the preceding slot) are assigned to their designated rear-stage satellites. Second, the remaining capacity is allocated by solving:

$$\begin{aligned} \min_{\alpha_{is}^t, x_{ism}^t} \quad & \sum_s Q_s^E(t) \cdot e_s^t + \sum_s Q_s^P(t) \cdot p_s^t \\ & - \sum_i (Q_i^U(t) + V) \cdot q_i^t \\ \text{s.t.} \quad & x_{ism}^t \leq y_{sm}, \quad \forall i, s, m, \\ & \alpha_{is}^t \cdot D_i \leq C_{is}^t \cdot \tau, \quad \forall i, s, \\ & \sum_i z_{ism}^t \leq 1, \quad \forall s, m, \\ & x_{ism}^t \leq \sum_{k=1}^t \alpha_{is}^k, \quad \forall i, s, m \in \mathcal{M}_S \cup \mathcal{M}_F, \\ & (15)-(16), \\ & \sum_{i,m} z_{ism}^t \hat{P}_{im}^{\text{CP}} \leq P_s^{\text{cap}}, \quad \forall s, \\ & e_s^t \leq b_s^t + \omega_s^t, \quad \forall s. \end{aligned} \quad (35)$$

The objective (35) is obtained by dropping the constant terms $\sum_i Q_i^U(t) \cdot a_i^t U^{\text{tgt}}$ from the drift-plus-penalty bound. Let $\mathcal{I}_t = \{i \in \mathcal{I} : a_i^t = 1\}$ denote the set of sources with pending tasks at slot t . When deciding to start a front stage on satellite s_1 , the scheduler verifies that an ISL-connected satellite s_2 with the corresponding rear-stage deployment will be available at slot $t + N_{im}$; this check uses deterministic orbital data and the current execution state.

Note that the instantaneous power cap (25) and battery feasibility appear as *hard* per-slot constraints, not only as virtual queues. The power cap must hold in every slot (it is a physical hardware limit), so the virtual queue $Q_s^P(t)$ provides additional back-pressure while the hard constraint guarantees per-slot compliance. The battery constraint $e_s^t \leq b_s^t + \omega_s^t$ prevents scheduling tasks that would deplete the battery below zero.

The objective has an intuitive interpretation. Each source i receives an effective scheduling weight $Q_i^U(t) + V$: the baseline weight V rewards quality throughput, while the deficit queue backlog $Q_i^U(t)$ provides additional priority to sources whose quality has fallen behind, thus enforcing max-min fairness. On the energy side, large $Q_s^E(t)$ discourages energy-intensive tasks on satellite s , and large $Q_s^P(t)$ discourages high-power tasks.

E. Robust Extension for Eclipse Periods

A fundamental challenge in solar-powered LEO satellites is the eclipse period. During eclipse, $\omega_s^t = 0$, so the energy queue (28) grows monotonically as tasks consume battery energy. Traditional Lyapunov optimization requires $\mathbb{E}[\Delta(\mathbf{Q}(t)) | \mathbf{Q}(t)] \leq 0$ for all t , which is impossible during eclipse.

Following the robust Lyapunov framework of [22], we relax the drift condition. Instead of requiring the drift to be non-positive at every slot, we allow temporary positive drift during eclipse, provided that the overall trend of the Lyapunov function remains downward.

Assumption 1. *There exist constants $c > 0$ and a time-varying function $\nu(t)$ with $-1 < c < 1$ and $\sum_{t=0}^T |\nu(t)| \leq \xi(T+1) + S_1$ for some $\xi \geq 0$ and $S_1 > 0$, such that the conditional drift satisfies*

$$\mathbb{E}[\Delta(\mathbf{Q}(t)) | \mathbf{Q}(t)] \leq (c + \nu(t) - 1) \mathbb{E}[L(\mathbf{Q}(t)) | \mathbf{Q}(t)]. \quad (36)$$

The function $\nu(t)$ captures the temporary energy disruption during eclipse. During sunlit periods, $\nu(t) \approx 0$ and the drift is strictly negative. During eclipse, $\nu(t) > 0$ but bounded, so the drift may be temporarily positive. As long as the eclipse fraction satisfies $\xi + |c| < 1$, the Lyapunov function converges to a bounded region, guaranteeing long-term stability (see Section V).

F. Overall Algorithm

The complete algorithm is summarized in Algorithm 1.

The algorithm has two phases. Phase 1 runs once at the beginning and solves a small ILP for model deployment. Phase 2 runs online in each slot, solving a lightweight assignment problem based on the current virtual queue backlogs. The per-slot complexity is dominated by the assignment problem (35), which is much smaller than the offline formulation ($\mathbf{P}_{\text{off}}$).

Remark 1 (Practical Implementation). *After fixing the model deployment $\{y_{sm}\}$, the per-slot problem (35) involves at most $|\mathcal{I}_t| \cdot |\mathcal{S}|$ binary variables for standalone tasks plus $|\mathcal{I}_t| \cdot |\mathcal{S}|^2$ for pipeline satellite pairs. In practice, the pipeline terms are sparse: each front-stage satellite has at most four ISL*

Algorithm 1: Robust Lyapunov Online Scheduling for Energy-Aware Collaborative VLM Inference

Input: Satellite set $\mathcal{S}$, VLM configurations $\mathcal{M}$, energy parameter lookup table, control parameter V , quality target U^{tgt} .

Output: Per-slot scheduling decisions $\{\alpha_{is}^t, x_{ism}^t\}$.

- 1 **Phase 1: Offline Model Deployment**
 - 2 Solve (27) to obtain $\{y_{sm}^*\}$
 - 3 **Phase 2: Online Per-Slot Scheduling**
 - 4 Initialize $Q_s^E(0) = Q_s^P(0) = 0$ for all s ; $Q_i^U(0) = 0$ for all i
 - 5 **for** $t = 1, 2, \dots, T$ **do**
 - 6 Observe current state: pending tasks $\mathcal{I}_t$, arrivals $\{a_i^t\}$, channels $\{C_{is}^t\}$, ISL connectivity $\{\ell_{ss'}^t\}$, solar $\{E_s^t\}$, battery $\{b_s^t\}$
 - 7 Schedule mandatory rear stages (from pipelines whose front stages completed)
 - 8 Solve per-slot subproblem (35) for remaining capacity to obtain $\{\alpha_{is}^{t*}, x_{ism}^{t*}\}$
 - 9 Execute decisions: upload, schedule tasks, transfer activations
 - 10 Update virtual queues via (28), (29), (30)
 - 11 Update battery:

$$b_s^{t+1} = \min\{\max\{b_s^t + \omega_s^t - e_s^t, 0\}, B_s^{\max}\}$$
 - 12 **end**
-

neighbors, so the effective variable count is $O(|\mathcal{I}_t| \cdot |\mathcal{S}|)$. Rear-stage scheduling is deterministic once the front stage starts (the rear satellite and timing are fixed by the pipeline coupling and precedence constraints), so the online optimizer decides only whether to initiate a new standalone or pipeline task. A designated lead satellite or ground station collects virtual queue states via ISL and broadcasts scheduling decisions within a single slot.

V. THEORETICAL ANALYSIS

In this section, we analyze the performance of Algorithm 1. We first establish the drift bound constant, then prove virtual queue stability, and finally derive the optimality gap relative to the offline MILP benchmark ($\mathbf{P}_{\text{off}}$).

A. Drift Bound

Lemma 1 (Drift Bound Constant). *Under Algorithm 1, the one-slot conditional Lyapunov drift satisfies (33) with*

$$B = \frac{1}{2} \sum_{s \in \mathcal{S}} [(e_s^{\max})^2 + (\omega_s^{\max})^2 + (p_s^{\max})^2 + (P_s^{\text{cap}})^2] + \frac{|\mathcal{I}|}{2} \max\{(U^{\text{tgt}})^2, (Q_{\max})^2\}, \quad (37)$$

where $e_s^{\max} = \max_{i,m} \mathbb{E}[E_{im}]/N_{im}$ is the maximum per-slot energy consumption, $\omega_s^{\max} = E_s^{\max}$ is the maximum solar energy arrival, $p_s^{\max} = \max_{i,m} \hat{P}_{im}^{\text{CP}}$ is the maximum calibrated power, and $Q_{\max} = \max_{i,m} Q_{im}$ is the maximum inference quality score.

Proof. Squaring both sides of the virtual queue updates (28)–(30) and using the identity $(\max\{a+b, 0\})^2 \leq a^2 + b^2 + 2ab$ for all a, b , we bound the single-slot squared increment of each queue. For the quality queue, the maximum single-slot input is $\max\{U^{\text{tgt}}, Q_{\max}\}$ (since $a_i^t \in [0, 1]$). Summing over all queues and taking conditional expectations yields the result. The constant B is finite because all system parameters are bounded. $\square$

B. Virtual Queue Stability

Theorem 1 (Mean-Rate Stability). *Suppose there exists a stationary randomized policy π^* that satisfies all constraints of (P) with a strict slack $\epsilon > 0$:*

$$\mathbb{E}[e_s^t - \omega_s^t] \leq -\epsilon, \quad \mathbb{E}[a_i^t U^{\text{tgt}} - q_i^t] \leq -\epsilon, \quad \forall s, i. \quad (38)$$

Then under Algorithm 1, all virtual queues are mean-rate stable:

$$\lim_{T \rightarrow \infty} \frac{\mathbb{E}[Q_s^E(T)]}{T} = 0, \quad \forall s \in \mathcal{S}, \quad (39)$$

and analogously for $Q_s^P(t)$ and $Q_i^U(t)$. This implies that the long-term energy budget constraint (24) is satisfied, and each source's average quality meets the target U^{tgt} .

Proof. By the drift bound in Lemma 1 and the existence of the stationary policy satisfying (38), we substitute the stationary policy into the drift-plus-penalty bound. Since Algorithm 1 minimizes the right-hand side of (33) $-V \sum_i q_i^t$ over all feasible policies (mandatory rear stages are common to every feasible policy and thus do not affect the optimality of the remaining decisions), the algorithm's drift-plus-penalty is no larger than the stationary policy's:

$$\begin{aligned} \Delta(\mathbf{Q}(t)) - V \sum_i \mathbb{E}[q_i^t] \\ \leq B - \epsilon \sum_s Q_s^E(t) - \epsilon \sum_i Q_i^U(t) + V q_{\Sigma}^*, \end{aligned} \quad (40)$$

where $q_{\Sigma}^* = \sum_i \mathbb{E}[q_i^{*,t}]$ is the total quality under the stationary policy. Summing over $t = 0, \dots, T-1$, telescoping, and dividing by T :

$$\begin{aligned} \frac{\epsilon}{T} \sum_{t=0}^{T-1} \sum_s \mathbb{E}[Q_s^E(t)] + \frac{\epsilon}{T} \sum_{t=0}^{T-1} \sum_i \mathbb{E}[Q_i^U(t)] \\ \leq B + V q_{\Sigma}^* + \frac{L(\mathbf{Q}(0))}{T}. \end{aligned} \quad (41)$$

Taking $T \rightarrow \infty$ establishes that the time-averaged backlogs are bounded, which implies mean-rate stability by [39]. $\square$

C. Optimality Gap

Theorem 2 (Optimality Gap). *Let q_{Σ}^* denote the optimal total time-average quality $\sum_i \bar{q}_i^*$ achievable by any policy satisfying all constraints of (P), and let $\bar{q}_{\Sigma}^{\text{ALG}} = \sum_i \bar{q}_i^{\text{ALG}}$ denote the total achieved by Algorithm 1. Then*

$$q_{\Sigma}^* - \bar{q}_{\Sigma}^{\text{ALG}} \leq \frac{B}{V}, \quad (42)$$

where B is the drift bound constant from Lemma 1. Moreover, mean-rate stability of the quality queues (Theorem 1) guarantees $\bar{q}_i^{\text{ALG}} \geq \lambda_i U^{\text{tgt}}$ for all i , so the minimum per-source quality satisfies $\min_i \bar{q}_i^{\text{ALG}} \geq \lambda_{\min} U^{\text{tgt}}$.

Proof. From the drift-plus-penalty bound, Algorithm 1 minimizes the right-hand side of (33) $-V \sum_i q_i^t$ in each slot. Substituting the optimal stationary policy π^* as a comparison:

$$\Delta(\mathbf{Q}(t)) - V \sum_i \mathbb{E}[q_i^{\text{ALG},t}] \leq B - V \sum_i \mathbb{E}[q_i^{*,t}]. \quad (43)$$

Summing from $t = 0$ to $T-1$ and using $L(\mathbf{Q}(T)) \geq 0$:

$$-V \sum_{t=0}^{T-1} \sum_i \mathbb{E}[q_i^{\text{ALG},t}] \leq T(B - V q_{\Sigma}^*) + L(\mathbf{Q}(0)). \quad (44)$$

Dividing by VT and taking $T \rightarrow \infty$ yields $\bar{q}_{\Sigma}^{\text{ALG}} \geq q_{\Sigma}^* - B/V$. The per-source bound follows from the quality queue stability condition (38). $\square$

Theorem 2 shows that the total quality gap vanishes as $V \rightarrow \infty$, while the quality virtual queues enforce per-source fairness with target U^{tgt} . However, a larger V leads to larger virtual queue backlogs (by Theorem 1, the average backlog scales as $O(V)$), meaning longer transient periods before constraints are satisfied. This is the classical $[O(1/V), O(V)]$ trade-off in Lyapunov optimization [39]. By searching over U^{tgt} , we can tune the max-min quality level.

D. Robust Stability Under Eclipse

Theorem 3 (Robust Stability). *Under Assumption 1, suppose the eclipse fraction parameter satisfies $\xi + |c| < 1$. Then the Lyapunov function is bounded:*

$$\limsup_{T \rightarrow \infty} \mathbb{E}[L(\mathbf{Q}(T))] \leq L^*, \quad (45)$$

where L^ is a finite constant that depends on $B, V, q_{\Sigma}^*, c, \xi$, and the maximum eclipse duration. This implies that all virtual queues remain bounded in expectation even under intermittent eclipse-induced energy disruptions.*

Proof. The proof follows the discrete-time ISS Lyapunov argument of [22]. Define $X_n = \mathbb{E}[L(\mathbf{Q}(n))]$. From Assumption 1:

$$X_{n+1} \leq (c + \nu(n))X_n + B + V q_{\Sigma}^*. \quad (46)$$

Since $|c| + \xi < 1$, the product $\prod_{k=\lfloor n/2 \rfloor}^{n-1} (|c| + |\nu(k)|)$ converges to zero as $n \rightarrow \infty$ (by the Gronwall-type inequality in [22], Lemma 1). Therefore, $\limsup_{n \rightarrow \infty} X_n$ is bounded by a constant proportional to $(B + V q_{\Sigma}^*)/(1 - |c| - \xi)$. $\square$

Theorem 3 is the key theoretical result that distinguishes our framework from standard Lyapunov approaches. During eclipse, the energy virtual queue $Q_s^E(t)$ grows because no solar energy is harvested. Traditional Lyapunov analysis would conclude instability. Our robust analysis shows that as long as the sunlit periods are sufficiently long relative to eclipse (captured by $\xi + |c| < 1$), the queues recover after each eclipse and remain bounded over time. For a typical LEO orbit at 550 km altitude with a roughly 60%/40% sunlit/eclipse split, this condition is satisfied.

VI. PERFORMANCE EVALUATION

VII. CONCLUSION

This paper presented the first online framework for energy-aware collaborative VLM inference over solar-powered LEO satellite networks. By formulating a joint optimization of two-stage pipeline splitting, task scheduling, and energy management under instantaneous power caps and orbital eclipse dynamics, we addressed four structural constraints that render existing collaborative inference and energy-harvesting schedulers inapplicable to the LEO setting. Our two-timescale robust Lyapunov framework decomposes the intractable long-term stochastic problem into an offline ILP for model deployment and a lightweight per-slot online scheduler, achieving an $O(1/V)$ optimality gap with provable mean-rate constraint satisfaction and robust stability during eclipse via an ISS-based analysis.

Several directions merit further investigation. First, incorporating adaptive activation compression [35] into the pipeline formulation could further reduce ISL bandwidth requirements, especially when laser links are congested. Second, extending the framework to multi-hop pipeline execution (three or more stages) would accommodate even larger VLMs at the cost of additional precedence constraints. Third, the robust Lyapunov analysis could be tightened by exploiting the deterministic periodicity of LEO eclipse patterns rather than treating eclipse as a generic bounded disturbance.

Finally, while this paper focuses on LEO Earth-observation constellations, the proposed framework naturally extends to *cislunar* satellite networks. Planned lunar constellations such as ESA Moonlight and NASA LunarNet will orbit the Moon for surface monitoring and communication relay. In this setting, the eclipse constraints become significantly more severe: Earth-shadow lunar eclipses can last several hours, far exceeding the ~ 35 -minute LEO eclipse. Moreover, the 1.3-second one-way Earth–Moon communication latency makes on-board VLM inference the only viable option for time-sensitive lunar surface analysis, strengthening the motivation for collaborative on-orbit inference. Adapting the robust Lyapunov framework to handle these longer and less predictable eclipse intervals is an important direction for future work.

REFERENCES

- [1] B. Denby and B. Lucia, “A comprehensive survey on orbital edge computing: Systems, applications, and algorithms,” *arXiv preprint arXiv:2306.00275*, 2023.
- [2] G. Furano, G. Meoni, A. Dunne, D. Moloney *et al.*, “Towards the use of artificial intelligence on the edge in space systems: Challenges and opportunities,” *IEEE Aerosp. Electron. Syst. Mag.*, vol. 35, no. 12, pp. 44–56, Dec. 2020.
- [3] Others, “Enabling near-realtime remote sensing via satellite-ground collaboration of large vision-language models,” *arXiv preprint arXiv:2510.24242*, 2025.
- [4] G. Giuffrida, L. Diana, F. De Gioia, G. Benelli, G. Meoni, M. Donati, and L. Fanucci, “CloudScout: A deep neural network for on-board cloud detection on hyperspectral images,” in *Remote Sensing*, vol. 12, no. 14, 2020, p. 2230.
- [5] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An image is worth 16x16 words: Transformers for image recognition at scale,” in *Proc. ICLR*, 2021.
- [6] M. Dehghani, J. Djolonga, B. Mustafa, P. Padlewski, J. Heek, J. Gilmer, A. Steiner, M. Caron, R. Geirhos, I. Alabdulmohsin, R. Jenatton, L. Beyer, M. Tschannen, A. Arnab, X. Wang, C. Riquelme, M. Minderer, J. Puigcerver, U. Evci, M. Kumar, S. van Steenkiste, G. F. Elsayed, A. Mahendran, F. Yu, A. Oliver, F. Huot, J. Bastings, M. P. Collier, A. Gritsenko, V. Birodkar, C. Vasconcelos, Y. Tay, T. Leung, A. Keysers, J. Uszkoreit, L. Bengio, N. Houlsby, and X. Zhai, “Scaling vision transformers to 22 billion parameters,” *arXiv preprint arXiv:2302.05442*, 2023.
- [7] R. Reedha, S. Drefahl, A. Boulch, and S. Lefèvre, “On-board satellite image classification for earth observation: A comparative study of pre-trained vision transformer models,” *arXiv preprint arXiv:2409.03901*, 2024.
- [8] Z. Lyu, M. Xiao, J. Xu, M. Skoglund, and M. D. Renzo, “The larger the merrier? efficient large AI model inference in wireless edge networks,” *IEEE J. Sel. Areas Commun.*, vol. 44, no. 10, pp. 2839–2853, 2025.
- [9] G. Xu, Z. Hao, Y. Luo, H. Hu, J. An, and S. Mao, “DeViT: Decomposing vision transformers for collaborative inference in edge devices,” *IEEE Trans. Mobile Comput.*, vol. 23, no. 5, pp. 5917–5932, 2024.
- [10] P. Dai, B. Han, K. Li, X. Xu, H. Xing, and K. Liu, “Joint optimization of device placement and model partitioning for cooperative DNN inference in heterogeneous edge computing,” *IEEE Trans. Mobile Comput.*, vol. 24, no. 1, pp. 210–226, 2025.
- [11] S. Zhang, W. Wu, L. Li, Y. Wang, and X. S. Shen, “Communication-efficient collaborative LLM inference over LEO satellite networks,” *arXiv preprint arXiv:2604.04654*, 2025.
- [12] Others, “Unseen cost of space computing: Quantifying LEO battery aging via physics-driven modeling,” *arXiv preprint arXiv:2603.04372*, 2025.
- [13] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, “A survey on mobile edge computing: The communication perspective,” *IEEE Commun. Surveys Tuts.*, vol. 19, no. 4, pp. 2322–2358, 2017.
- [14] J. Mei, L. Dai, Z. Tong, X. Deng, and K. Li, “Throughput-aware dynamic task offloading under resource constraint for MEC with energy harvesting devices,” *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 4, pp. 4697–4711, 2023.
- [15] A. Khoshshirat, G. Perin, and M. Rossi, “Decentralized LLM inference over edge networks with energy harvesting,” in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, 2024, pp. 3703–3708.
- [16] X. Yi, K.-W. Chin, Z. Li, and K. Liu, “Free space optical links scheduling and routing in satellite networks: A safe reinforcement learning approach,” *IEEE Internet Things J.*, vol. 12, no. 14, pp. 25 712–25 721, Jul. 2025.
- [17] Y. Luo, L. Pu, and C.-H. Liu, “Computing power and battery charging management for solar energy powered edge computing,” *IEEE Trans. Mobile Comput.*, vol. 24, no. 3, pp. 1913–1927, 2025.
- [18] S. Duan *et al.*, “EdgeShard: Efficient LLM inference via collaborative edge computing,” *arXiv preprint arXiv:2405.14371*, 2024.
- [19] Z. Li, W. Feng, M. Guizani, and H. Yu, “Tpi-LLM: Serving 70b-scale LLMs efficiently on low-resource mobile devices,” *IEEE Trans. Serv. Comput.*, pp. 1–13, 2025.
- [20] D. Xu, W. Yin, H. Zhang, X. Jin, Y. Zhang, S. Wei, M. Xu, and X. Liu, “EdgeLLM: Fast on-device LLM inference with speculative decoding,” *IEEE Trans. Mobile Comput.*, vol. 24, no. 4, pp. 3256–3273, 2025.
- [21] Y. He, J. Fang, F. R. Yu, and V. C. M. Leung, “Large language models (LLMs) inference offloading and resource allocation in cloud-edge computing: An active inference approach,” *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 11 253–11 264, 2024.
- [22] Z. Huang, Z.-P. Jiang, Z. Han, and Y. Liu, “Robust Lyapunov optimization for LEO satellite networks routing control,” *IEEE Trans. Mobile Comput.*, vol. 24, no. 12, pp. 13 293–13 307, Dec. 2025.
- [23] P. Helber, B. Bischke, A. Dengel, and D. Borth, “EuroSAT: A novel dataset and deep learning benchmark for land use and land cover classification,” *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 12, no. 7, pp. 2217–2226, 2019.
- [24] G. Cheng, J. Han, and X. Lu, “Remote sensing image scene classification: Benchmark and state of the art,” *Proc. IEEE*, vol. 105, no. 10, pp. 1865–1883, 2017.
- [25] L. Cheng, G. Feng, Y. Sun, S. Qin, F. Wang, and T. Q. S. Quek, “Energy-constrained satellite edge computing for satellite-terrestrial integrated networks,” *IEEE Trans. Veh. Technol.*, vol. 74, no. 3, pp. 3359–3374, 2025.
- [26] S. Yao, Y. Lin, M. Wang, K. Xu, M. Xu, C. Xu, and H. Zhang, “LEOEdge: A satellite-ground cooperation platform for the AI inference in large LEO constellation,” *IEEE J. Sel. Areas Commun.*, vol. 43, no. 1, pp. 36–50, 2025.

- [27] M. Sun, J. Hou, K. Qiu, K. Wang, X. Chu, and Z. Zhang, “LLM-based task offloading and resource allocation in satellite edge computing networks,” *IEEE Trans. Veh. Technol.*, vol. 75, no. 4, pp. 5262–5267, 2026.
- [28] Q. Tang, R. Xie, Z. Fang, T. Huang, T. Chen, R. Zhang, and F. R. Yu, “Joint service deployment and task scheduling for satellite edge computing: A two-timescale hierarchical approach,” *IEEE J. Sel. Areas Commun.*, vol. 42, no. 5, pp. 1063–1079, 2024.
- [29] J. Zhou, Q. Yang, L. Zhao, H. Dai, and F. Xiao, “Mobility-aware computation offloading in satellite edge computing networks,” *IEEE Trans. Mobile Comput.*, vol. 23, no. 10, pp. 9135–9149, 2024.
- [30] G. Zheng, Q. Ni, K. Navaie, and H. Pervaiz, “Semantic communication in satellite-borne edge cloud network for computation offloading,” *IEEE J. Sel. Areas Commun.*, vol. 42, no. 5, pp. 1145–1158, 2024.
- [31] Y. Chen, J. Zhao, Y. Wu, J. Huang, and X. Shen, “Multi-user task offloading in UAV-assisted LEO satellite edge computing: A game-theoretic approach,” *IEEE Trans. Mobile Comput.*, vol. 24, no. 1, pp. 363–378, 2025.
- [32] J. Wu, M. Jia, and Q. Guo, “Dynamic caching-assisted inter-satellite cooperative computation offloading in LEO satellite edge computing,” *IEEE Trans. Veh. Technol.*, vol. 75, no. 2, pp. 2749–2761, 2026.
- [33] Q. He, Y. Xia, Z. Feng, L. Ma, Y. Lv, K. Yu, A. Hawbani, K. Zheng, and L. Xu, “Computation resource management in mobile edge computing for healthcare using Lyapunov-deep deterministic policy gradient,” *IEEE Trans. Mobile Comput.*, vol. 25, no. 2, pp. 2159–2173, Feb. 2026.
- [34] M.-L. Ku, Y. Chen, and K. J. R. Liu, “Data-driven stochastic models and policies for energy harvesting sensor communications,” *IEEE J. Sel. Areas Commun.*, vol. 33, no. 8, pp. 1505–1520, Aug. 2015.
- [35] Others, “Why should the server do it all?: A scalable, versatile, and model-agnostic framework for server-light DNN inference via training-free intermediate feature compression,” *arXiv preprint arXiv:2511.11608*, 2025.
- [36] M. Kasioulis, M. Symeonides, G. Ioannou, G. Pallis, and M. D. Dikaiakos, “Energy modeling of inference workloads with AI accelerators at the edge: A benchmarking study,” in *Proc. IEEE IC2E*, 2024, pp. 189–196.
- [37] E. J. Husom, M. Salhov, M. Astekin, L. K. Shar, A. Kasen, B. Sen, and A. Soyul, “Sustainable LLM inference for edge AI: Evaluating quantized LLMs for energy efficiency, output accuracy, and inference latency,” *arXiv preprint arXiv:2504.03360*, 2025.
- [38] H. Xu, L. Peng, X. Song, M. Liu, S. Jun, J. Li, J. Yu, and X. Mao, “CAMEL: Energy-aware LLM inference on resource-constrained devices,” *arXiv preprint arXiv:2508.09173*, 2025.
- [39] M. J. Neely, “Stochastic network optimization with application to communication and queueing systems,” *Morgan & Claypool Publishers*, 2010.
- [40] L. Georgiadis, M. J. Neely, and L. Tassiulas, “Resource allocation and cross-layer control in wireless networks,” *Found. Trends Netw.*, vol. 1, no. 1, pp. 1–144, 2006.
- [41] R. Urgaonkar, S. Wang, T. He, M. Zafer, K. Chan, and K. K. Leung, “Dynamic service migration and workload scheduling in edge-clouds,” *Perform. Eval.*, vol. 91, pp. 205–228, 2015.

TABLE II
A SUMMARY OF NOTATIONS.

1.	Sets
$\mathcal{T}$	The set of discrete time slots, $ \mathcal{T} $ is the planning horizon.
$\mathcal{S}$	The set of computing satellites.
$\mathcal{I}$	The set of sensing sources (sensing satellites).
$\mathcal{M}$	The set of pre-enumerated VLM configurations ($\mathcal{M} = \mathcal{M}_S \cup \mathcal{M}_F \cup \mathcal{M}_R$).
$\mathcal{M}_S, \mathcal{M}_F, \mathcal{M}_R$	Standalone, front-stage, and rear-stage configuration subsets.
$\pi(m)$	Bijection mapping front stage $m \in \mathcal{M}_F$ to its rear stage in $\mathcal{M}_R$.
2.	Parameters (offline-computed constants)
τ	Duration of a time slot.
λ_i	Task arrival rate of source i .
a_i^t	Task arrival indicator: $a_i^t = 1$ if source i has a pending task at slot t .
D_i	Data size of a task from source i (in bits).
Q_{im}	Inference quality of source i under configuration m .
δ_s^t	Sunlit indicator: $\delta_s^t = 1$ if satellite s is sunlit at slot t , 0 during eclipse.
$\mathbb{E}[E_{im}]$	Expected inference energy of source i under configuration m .
$\mathbb{E}[T_{im}]$	Expected inference time of source i under configuration m .
N_{im}	Number of slots required: $N_{im} = \lceil \mathbb{E}[T_{im}]/\tau \rceil$.
$\hat{P}_{im}^{\text{CP}}$	Calibrated power upper bound for source i under configuration m .
P_s^{cap}	Hardware instantaneous power cap of satellite s .
B_s^{max}	Battery capacity of satellite s .
R_s^{max}	Memory (RAM) capacity of satellite s .
R_m	Memory (RAM) requirement of VLM configuration m .
L_m	Number of front-stage layers in split config $m \in \mathcal{M}_F$.
E_s^t	Solar energy arrival at satellite s in slot t .
ω_s^t	Energy actually stored: $\omega_s^t = \min(E_s^t, B_s^{\text{max}} - b_s^t)$.
C_{is}^t	ISL capacity between source i and satellite s in slot t ($= 0$ if no path).
$\ell_{ss'}^t$	ISL connectivity indicator between computing satellites s and s' at slot t .
d_{is}^t	Inter-satellite distance from source i to satellite s in slot t .
P_{tx}	Transmit power.
$G_{\text{tx}}, G_{\text{rx}}$	Transmit and receive antenna gains.
f	Carrier frequency.
$k_B T_{\text{sys}}$	System noise power spectral density.
W	Bandwidth of the inter-satellite link.
3.	Decision Variables
α_{is}^t	Whether source i 's task is uploaded to satellite s in slot t (binary).
y_{sm}	Whether satellite s is configured with VLM configuration m (binary).
x_{ism}^t	Whether source i 's task starts execution on (s, m) in slot t (binary).
4.	Derived / State Variables
z_{ism}^t	Execution indicator: source i executing on (s, m) in slot t (from x via (11)).
b_s^t	Battery level of satellite s at the start of slot t .
q_i^t	Inference quality delivered to source i in slot t : $q_i^t = \sum_{s,m} x_{ism}^t Q_{im}$.
$\bar{q}_i$	Long-term average quality of source i : $\bar{q}_i = \lim_{T \rightarrow \infty} \frac{1}{T} \sum_t \mathbb{E}[q_i^t]$.